```
24 * These changes are either USB devices or busses being added or
removed.
25 */
26 void usb_register_notify (struct notifier_block *nb)
27 {
28      blocking_notifier_chain_register (&usb_notifier_list, nb);
29}
30 EXPORT_SYMBOL_GPL(usb_register_notify);
```

The first step of the publisher is to provide a notifier list. The *usb_notifier_list* is declared as the notifier list head for the USB notification. An interface function that exports the USB notification list is also provided by the USB core. It is a better programming practice to provide an interface function than exporting a global variable.

Now, we can see how to write an example USB hook module that 'subscribes' to the USB core. The first step is to declare a handler function and initialise it to a *notifier_block* type variable. In the following example (a sample *usbhook.c* file), *usb_notify* is the handler function and it is initialised to a *notifier_block* type variable *usb_nb*.

```
/*
 * usbhook.c - Hook to the usb core
 */
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/usb.h>
#include <linux/notifier.h>

static int usb_notify(struct notifier_block *self, unsigned long action, void
*dev)
{
        printk(KERN_INFO "USB device added \n");
        switch (action) {
        case USB_DEVICE_ADD:
                printk(KERN_INFO "USB device added \n");
                break;
        case USB_DEVICE_REMOVE:
                printk(KERN_INFO "USB device removed \n");
                break;
        case USB_BUS_ADD:
                printk(KERN_INFO "USB Bus added \n");
                break;
        case USB_BUS_REMOVE:
                printk(KERN_INFO "USB Bus removed \n");
        }
        return NOTIFY_OK;
}
static struct notifier_block usb_nb = {
        .notifier_call =        usb_notify,
};
int init_module(void)
{
        printk(KERN_INFO "Init USB hook.\n");
        /*
* Hook to the USB core to get notification on any addition or removal of
USB devices
        */
        usb_register_notify(&usb_nb);

        return 0;
}

void cleanup_module(void)
{
        /*
         * Remove the hook
        */
        usb_unregister_notify(&usb_nb);

        printk(KERN_INFO "Remove USB hook\n");
}

MODULE_LICENSE("GPL");
```

The above sample code registers the *notifier_block usb_nb* using the interface function of the USB core. The interface function adds the *usbhook* function to the *usb_notifier_list* of the USB core. Now we are set to receive the notification from the USB core.

When a USB device is attached to the kernel, the USB core detects it and uses the *blocking_notifier_call_chain* API to call the registered subscribers:

```
60 void usb_notify_add_bus(struct usb_bus *ubus)
61 {
62   blocking_notifier_call_chain(&usb_notifier_list,
USB_BUS_ADD, ubus);
63 }
```

This *blocking_notifier_call_chain* will call the *usb_notify* function of the USB hook registered in *usb_notifier_list*.

The above sample briefs you on the infrastructure of Linux blocking notifier chains. The same methodology can be used for other types of notifier chains.

Linux kernel modules are loosely coupled and get loaded and unloaded runtime with ease. An effective methodology is required to communicate between these modules. Linux notifier chains do this effectively. They are mainly brought in for network devices and can be effectively used by other technologies as well. As developers, we have to look for such utility functions that are available in the kernel and use them effectively in our designs instead of reinventing the wheel. END

By: Rajaram Regupathy.
The author welcomes your comments and feedback at rera_raja@yahoo.com